

JARVIS Codebase Audit — Streaming Checkpoint

Baseline: ARCHITECTURE.md v3.1 (14 May 2026) | Audit date: 16 May 2026 | Auditor: Claude Opus 4.7 | Repo: jarvis @ main (1f8b762)

Executive Summary

Since the first audit (16 May, commit afb35f2), ten commits have landed and most of the prior P0/P1 list is closed. The provider layer is now a registry of ChatProvider implementations; the route validates input with Zod, enforces an in-memory rate limit, gates calls through a cost guard, loads the system prompt with a content hash, records a telemetry event on success/failure, and streams the model output back as text/plain through a ReadableStream. The client consumes the stream incrementally and renders tokens as they arrive.

This checkpoint asks a different question: is that streaming layer ready to take Anthropic, sentence chunking, TTS, tool-calls, and the SSE/event multiplexing v3.1 implies — or does it lock in technical debt? The answer is: **the abstraction is the right shape for token streaming today, but the wire format (text/plain) and the stream payload type (AsyncIterable<string>) are both one-channel choices that the architecture's next four phases will outgrow.** Fix the payload type and switch the wire to SSE (or NDJSON) before Anthropic lands. Both are small, local changes today; both are large, cross-cutting refactors after a second provider exists.

Verdict: The Phase 1.5 gate proposed in the prior audit is now ~80% done. Two streaming-shaped fixes (typed stream events + SSE wire format) remain before Phase 1B (Anthropic) can land without re-opening every consumer.

1. What Changed Since the Prior Audit

Ten commits, all in the direction of the prior fix list. Status of the prior P0/P1 items:

#	Prior P0/P1 item	Status now	Evidence
1	Rotate exposed API keys	OUT OF SCOPE for this audit	Use a side action; cannot verify from repo
2	Cost guard with daily/weekly/monthly caps	DONE	src/lib/cost/* — guard.ts checks each window before request; usage.record
3	Zod validation on /api/chat	DONE	ChatRequestSchema in route.ts; safeParse + 400 with issues
4	Rate limit on /api/chat	DONE	src/lib/rate-limit/* — 20 req / 60s sliding window, keyed by x-forwarded-f
5	Client error path (don't render error as assistant response)	DONE	app/errors checks res.ok before reading the body and renders a separate re
6	Fail fast if API keys missing	PARTIAL	config.ts still uses "" pattern; no boot-time assertion
7	System prompt under prompts/ with content hash logging	DONE	prompts/jarvis_system.md exists; loader hashes content and route logs the has
8	Stack decision (TS-first vs Python-first)	PENDING	TS-first is the de facto answer; ARCHITECTURE.md still claims Python &mdas
9	ChatProvider interface	DONE	src/lib/providers/types.ts — generate() and stream() on a single interface
10	Refactor openai.ts to implement ChatProvider interface	DONE	OpenAIProvider class; registered in registry.ts
11	Streaming from route, incrementally with caveats (see Section 2)	DONE	ReadableStream in route; getReader() + TextDecoder in client

#	Prior P0/P1 item	Status now	Evidence
12	SQLite telemetry per v3.1 §17	PARTIAL	In-memory store with the right shape but missing fields and not persisted; ring-
13	Registry-driven model ids	PARTIAL	Registry exists for providers, but model strings still live in config.ts
14	Fix Anthropic model id (claude-opus-1 → currently)	OPEN	config.ts still says claude-opus-4-1
15	Cap conversation length / summarize N turns	OPEN	Client still re-uploads full history each turn
16	Persist sessions to SQLite, not use state	OPEN	Still client-only state
17	tsconfig path mapping	DONE	Now @/* → ./src/* — consistent with conventions

2. Streaming Architecture — What's in the Repo Today

2.1 Provider stream abstraction

src/lib/providers/types.ts defines:

```
export interface StreamResult {
  stream: AsyncIterable<string>;
  done: Promise<GenerateResult>;
}

export interface ChatProvider {
  readonly id: ProviderId;
  generate(messages: Message[], opts: GenerateOptions): Promise<GenerateResult>;
  stream(messages: Message[], opts: GenerateOptions): Promise<StreamResult>;
}
```

OpenAIProvider.stream() creates an SDK stream with stream_options: { include_usage: true }, returns an async generator that yields delta.content strings, and resolves done with the final GenerateResult (content, modelId, input/output tokens, costUsd, latencyMs) after the last chunk.

2.2 Route streaming

app/api/chat/route.ts wraps the provider stream in a ReadableStream<Uint8Array>, encodes each yielded string with TextEncoder, and returns it with Content-Type: text/plain; charset=utf-8, Cache-Control: no-cache, no-transform, X-Content-Type-Options: nosniff. After the iterator drains, the route awaits streamResult.done, records cost, and emits a model_call telemetry event. On error inside the iterator, it emits provider_error and calls controller.error().

2.3 Frontend streaming

app/page.tsx uses res.body.getReader() + TextDecoder({ stream: true }), appends each decoded chunk to assistantContent, and re-sets the messages array on every chunk to render the tail message progressively.

3. Streaming Findings — What Will Bite You at Phase 1B

3.1 The stream payload type cannot carry non-text events

AsyncIterable<string> is a single-channel pipe. Today, only token deltas flow through it. Tomorrow, all of the following need to flow through the same pipe:

- **Anthropic structured events** — message_start, content_block_start, content_block_delta, content_block_stop, message_delta, message_stop. The current provider would collapse all of these into bare strings, discarding the structure.
- **Tool-call streaming** (Anthropic tool_use blocks, OpenAI tool_calls deltas). These are not text. They have ids, names, and incrementally-streamed JSON args. A string stream cannot represent them.
- **Sentence boundaries** for v3.1 §8.1 TTS dispatch. Sentence-chunking lives *downstream* of the provider, but it needs an event saying "this is the end of a content block" — otherwise it can't flush the last sentence reliably.

- **Mid-stream errors with structured metadata** (rate-limit, model overloaded, safety stop). Today: `controller.error()` after HTTP 200 is already on the wire — the client sees a truncated reply with no error code.
- **Heartbeats / status** ("thinking", "using tool X", "running RAG"). Useful for the UI; impossible without typed events.

Recommended shape (concrete enough to implement this week):

```
export type StreamEvent =
  | { type: 'text'; value: string }
  | { type: 'tool_call_start'; id: string; name: string }
  | { type: 'tool_call_delta'; id: string; argsJsonChunk: string }
  | { type: 'tool_call_end'; id: string }
  | { type: 'usage'; inputTokens: number; outputTokens: number }
  | { type: 'error'; message: string; recoverable: boolean }
  | { type: 'done'; result: GenerateResult };

export interface StreamResult {
  events: AsyncIterable<StreamEvent>;
  // 'done' Promise is optional now &mdash; the last event already carries the result.
}
```

Migration cost **today**: one provider, one route, one client. Migration cost **after Anthropic lands**: two providers, every consumer, both telemetry and TTS pipelines that haven't been built yet but were designed against the old shape. The size of this change roughly doubles per phase.

3.2 text/plain is the wrong wire format for what comes next

text/plain works for the current single-channel case. It cannot represent the typed events from 3.1, so as soon as the payload type changes, the wire has to change too. Three options, in increasing order of headroom:

Option	How it looks	Pros	Cons
NDJSON	One JSON object per line: <code>{"type": "text", "text": "Hello!"}</code>	Trivial to parse; text-readable	Not a native HTTP feature; requires a JSON parser on the client
SSE (text/event-stream)	<code>event: text\ndata: { "type": "text", "text": "Hello!" }</code>	Native HTTP feature; text-readable	Source format is not fully-featured; requires a parser on the client
WebSocket	Bidirectional frames	Bidirectional — useful when the client needs to send data back	Not a native HTTP feature; requires a WebSocket client on the client

Recommendation: SSE, with fetch + a hand-rolled parser on the client (same pattern the Anthropic and OpenAI SDKs use internally). It is the cheapest wire that carries typed events and survives Phase 4 voice streaming without another migration.

3.3 AbortSignal is not threaded anywhere

`provider.stream()` takes no `signal`. The route never watches `req.signal`. The client cannot cancel. The route also cannot detect a client disconnect and stop iterating — the OpenAI SDK will happily keep streaming tokens you'll pay for that nobody is reading. This is a real cost leak the moment a user closes the tab on a long generation.

- Add `signal?: AbortSignal` to `GenerateOptions`.
- Pass `req.signal` through to `provider.stream()`.
- Listen to `req.signal.addEventListener('abort', ...)` in the `ReadableStream`'s `cancel()` hook and propagate.

- Add a **Stop** button on the client with an `AbortController`.
- Record a `client_disconnect` telemetry event so cost-leak protection is observable.

3.4 No time-to-first-token captured

v3.1 §7 sets first-token P95 budgets per tier (T3: <1.5s; T4: <3s). Today the only latency recorded is end-to-end — the metric the architecture cares about most for streaming is unmeasured. Add `time_to_first_token_ms` to `GenerateResult` and to the telemetry event. Capture it inside the provider iterator on the first non-empty content delta.

3.5 Sentence-chunker has no home yet

v3.1 §8.1 puts sentence buffering between the LLM stream and the TTS stream. It is provider-agnostic but deeply stream-coupled. The right home is `src/lib/streaming/sentence_chunker.ts` — a transformer that consumes `AsyncIterable<StreamEvent>` and yields `AsyncIterable<SentenceEvent>`. This module does not need to exist before Phase 4 (voice), but the *event shape* it consumes has to be settled in 3.1's fix so the chunker can be added later without touching providers.

3.6 done Promise + iterable is awkward, not load-bearing

If a consumer breaks out of the `for-await` loop (e.g., on client disconnect), `done` never resolves, leaving a dangling Promise. Today the route never breaks out, so this is latent. Replacing it with a terminal `{type: 'done', result}` event removes the dangling-Promise shape and aligns with the SSE model (the stream is the source of truth; there is no out-of-band channel).

3.7 PLACEHOLDER_COST_PER_REQUEST_USD = 0.001

Cost is a flat \$0.001 per call regardless of token usage. The cost guard is therefore measuring something approximately unrelated to reality. Replace with a per-model rate from the registry, multiplied by actual `inputTokens / outputTokens` from the streamed usage chunk. Fixed-rate caps cannot enforce real spend — v3.1 §16 is unmet in spirit even though the code path exists.

4. Telemetry Compatibility

4.1 What works

- Stream success path records `model_call` with model id, latency, cost, and tokens (in the notes field).
- Failure paths record `validation_failure`, `rate_limited`, `cost_denied`, `provider_error`.
- Streamed usage from `stream_options.include_usage` is captured on the final chunk.

4.2 What diverges from v3.1 §17

v3.1 §17 specifies an SQLite events table with these fields:

```
id, ts, session_id, event_type, intent, safety_tag, tier, model_id,
tool_name, input_tokens, output_tokens, cost_usd, latency_ms, success,
error_class, user_rating, notes
```

Current `TelemetryEvent`:

```
timestamp, event_type, success, model_id?, latency_ms?, cost_usd?, error_class?, notes?
```

Missing or collapsed into notes:

- `id` — no event id; can't reference an event from elsewhere
- `session_id` — no notion of a session yet (also blocks v3.1 §15 memory)
- `intent`, `safety_tag`, `tier` — router fields, not relevant until Phase 1D, but the column should exist
- `tool_name` — not relevant yet, same note
- `input_tokens`, `output_tokens` — currently stuffed into notes as a free-text string; can't be queried
- `user_rating` — v3.1 §4.5 quality threshold depends on this; no UI for it either

4.3 What the streaming work specifically needs added

- `time_to_first_token_ms` — first-token latency, per §3.4 above
- `stream_outcome`: 'completed' | 'client_aborted' | 'error_before_first_token' | 'error_mid_stream' — distinguishes recoverable from unrecoverable failures
- `bytes_streamed` / `chunks_streamed` — helps debug throughput issues

4.4 Storage shape

`InMemoryTelemetryStore` is fine for development — the contract is `TelemetryStore` and can be swapped for a SQLite implementation without touching the route. The ring-buffer cap at 1000 silently drops oldest events; for now, add a console warning when trimming so this doesn't hide. Promotion to `better-sqlite3` can wait until Phase 1.5 has a `session_id` to write.

5. Smaller Issues Worth Picking Up

5.1 Config and identity

- `config.ts: anthropic.model = "claude-opus-4-1"` — still wrong (current is `claude-opus-4-7`). Same issue as the prior audit; pushed forward but never fixed. Easier still: delete this field, source from the registry once the registry holds model ids.
- `config.ts: process.env.OPENAI_API_KEY || ""` still in place. Fail-fast assertion at module load remains open.
- `app/layout.tsx: metadata.title = "Create Next App"` — still the default. One-line fix.
- Geist fonts loaded but `globals.css` still uses Arial. Either drop the font loading or apply the CSS variable to body.

5.2 Client UX

- **No Stop button.** Tied to §3.3 AbortSignal — one component, real value the moment generation gets slow.
- **No auto-scroll** as new tokens stream in; the visible content can fall off the bottom of the viewport.
- `loading=true` **stays true throughout the stream.** The "JARVIS is thinking..." placeholder shows alongside the streaming reply — flip to a 'streaming' state once the first token arrives so the placeholder hides.
- **Full** `setMessages([...])` **on every chunk.** Use an updater function (`setMessages(prev => ...)`) or move the streaming content into a ref to avoid re-rendering the entire list on every token.
- **React key={index}** — still a footgun the moment edit/regenerate exists. Assign a stable id at message creation.
- **Enter sends, Shift+Enter does not newline** — `<input>` instead of `<textarea>`.
- **No request size cap** — the route accepts arbitrarily large message arrays. Bound it to e.g. 50 messages × 4KB each in the Zod schema.

5.3 Tests & pre-commit (still open)

- No vitest or playwright in `package.json`; no tests on disk.
- No husky / lint-staged / Prettier pre-commit. v3.1 Phase 1 P1 requires this.
- Provider stream is the most subtle code in the repo and has zero unit tests. A mock-OpenAI stream test (yield three deltas + a usage chunk, assert events + final result) is <30 lines.

5.4 Docs reconciliation

- `ARCHITECTURE.md` still describes the Python/FastAPI/Ollama runtime in §10 and §20. The code says TypeScript/Next is the answer. Edit the architecture (in place, with a 'Stack' subsection at the top) so the two no longer disagree.
- `README` hasn't been re-read in this audit; the prior audit flagged a Python-narrative mismatch — check whether streaming/cost/rate-limit setup steps now match the code.

6. Should Anything Change Before Anthropic Lands?

Yes — two changes, both cheap now.

6.1 Must-do before Phase 1B

#	Change	Why now
A	Change <code>StreamResult.stream</code> from <code>AsyncIterable<string></code> to <code>AsyncIterable<string></code>	Adding an <code>AbortSignal</code> to <code>StreamResult</code> is a small change, but it's a good idea to have it now.
B	Switch <code>/api/chat</code> response from <code>string</code> to <code>StreamResult</code>	Switching the response type to <code>StreamResult</code> is a small change, but it's a good idea to have it now.
C	Thread <code>AbortSignal</code> from <code>StreamResult</code> to <code>StreamResult</code>	Adding an <code>AbortSignal</code> to <code>StreamResult</code> is a small change, but it's a good idea to have it now.
D	Add <code>time_to_first_token_ms</code> to <code>StreamResult</code>	Adding <code>time_to_first_token_ms</code> to <code>StreamResult</code> is a small change, but it's a good idea to have it now.

6.2 Nice-to-have but not blocking

#	Change	Why deferrable
E	Replace placeholder cost with per-model token-rate from a registry	Costs are already gates; absolute precision matters more after a while.
F	Promote telemetry to SQLite with the full v3.1 §17 schema.	In-memory is fine until sessions exist (which they don't yet).
G	Sentence-chunker scaffold under <code>src/lib/streaming</code>	Waiting until Phase 4 voice; the <code>StreamEvent</code> shape from (A).
H	Cap conversation history / summarise at N turns.	Real cost risk, but unrelated to the stream architecture. Defer to a later date.

6.3 Future-compat scorecard

Future capability	Compatible with current stream layer	If not, what changes
Anthropic text streaming	Partially — works as bare string	(A) typed events structure.
Anthropic tool-use streaming	No.	(A) typed events + <code>tool_call</code> event types.
OpenAI <code>tool_calls</code> streaming	No — route only reads <code>delta.tool_calls</code>	(A) typed events + <code>tool_calls</code> event types.
Sentence chunking for TTS	No clean home, but possible.	(A) typed events so the chunker can listen for content-block boundaries.
TTS streaming dispatch	No.	(A) + new streaming module; consumes events, dispatches audio.
SSE / event multiplexing	No — wire is text/plain.	(B) switch to text/event-stream.
Client cancel / disconnect	No.	(C) thread <code>AbortSignal</code> .
First-token latency budgets (§7)	Unmeasured.	(D) capture in iterator.

7. Closing

The streaming work that landed is good. Provider abstraction is the right idea in the right place. The route is well-structured (validate → rate-limit → cost-gate → stream → record). The client streams correctly and degrades sensibly on error. None of this needs to be undone.

What needs doing — once, this week — is widening the stream's *type* and *wire format* from one channel to many, before a second provider arrives and locks both choices in. Items A–D in §6.1 are the smallest change that makes that happen. Everything else in this audit can wait.

— Audit complete.